Project CampaignX

Automating SuperBFSI Marketing

An AI Multi-Agent System built for FrostHack 2026

The Core Problem

Digital marketing at SuperBFSI is heavily manual. Campaign managers waste hours on data gathering, segmentation, scheduling, and pulling reports instead of focusing on actual strategy. A/B testing is often skipped because it takes too long to set up.

Our Solution

We built an autonomous multi-agent system that takes a plain-text prompt, reads the OpenAPI specs to figure out the marketing tools, drafts the emails, and schedules the campaigns.

> **Crucial Rule:** The system is autonomous, but not reckless. It pauses execution and waits for a human to review the draft and target list via a web UI before making any API calls to send the emails.

System Architecture

We designed the system to be modular so the AI agents can talk to the database, the external APIs, and the frontend without tripping over each other.

→ **The Orchestrator (Backend):** Built with Python and FastAPI. This manages the state of the campaign and

Data Flow

```
[User Prompt] -> (Frontend UI) | v [FastAPI Backend] <---> [PostgreSQL DB] | [Agent Orchestrator] ├──> Plann
```

Key Features Built Into The Model

1. Natural Language Brief Parsing

The user types: *"Send an email about XDeposit to inactive users, add 0.25% for seniors."* The model extracts the product name, parses the target audience (mapping 'inactive' to our DB flags), and embeds the specific business rules into the prompt for the writer agent.

2. Dynamic API Discovery

Instead of hardcoding the POST requests, we feed the OpenAPI specification to the execution agent. It reads the docs dynamically to figure out which endpoint to hit, what headers are needed, and how to format the payload.

3. Smart Content Generation & A/B Testing

The model doesn't just write one email. It generates variations in tone (formal vs. casual), decides if bolding specific text improves readability, and programmatically injects the required tracking URLs. It splits the cohort to test these variations.

4. Autonomous Optimization Loop

Once a campaign is running, a chron job triggers the Monitoring Agent. It fetches the latest Open and Click rates. If Variation B is outperforming Variation A, it automatically flags Variation A to be

How The Agentic Loop Actually Runs

This is the step-by-step logic the backend executes when a new campaign is initiated.

1. **Ingest & Strategize**

   The Planner Agent queries the Customer Cohort API to get the master list. It filters the IDs based on the marketer's prompt to create the target segment.

2. **Draft & Stage**

   The Writer Agent creates the email assets. The Orchestrator saves everything to the database with a status of `PENDING_APPROVAL`. The system completely pauses execution here.

3. **Human-in-the-Loop Override**

   The marketer logs into the UI, reviews the drafted emails, the segmented customer list, and the scheduled send time. They click "Approve".

4. **Dispatch & Monitor**

Technical Hurdles & How We Solved Them

Handling the "Pause" State

The Issue: Standard LLM chains want to run from start to finish. Forcing an AI agent to stop, wait for a human on a web browser, and pick up exactly where it left off was breaking the execution loop.

The Fix: We decoupled the process. The planning and drafting agents run synchronously and dump their payload into the database. A completely separate execution agent listens for a database webhook (when status changes to 'Approved') to handle the actual API dispatch.

Preventing Hallucinated API Calls

The Issue: When using dynamic tool discovery, early versions of the model would occasionally guess the endpoint URLs or send malformed JSON bodies that the API rejected.

The Fix: We implemented strict Pydantic schemas for the tool inputs. Before the agent is allowed to make an HTTP request, the payload is validated against the schema. If it fails, the error is fed back to the LLM automatically so it can correct its own JSON formatting before trying again.